

JavaScript Day 1 - Revision Notes

Table of Contents

1. Introduction to Web Development
 2. Browser Console & REPL
 3. Variables in JavaScript
 4. Data Types
 5. Arithmetic Operations
 6. NaN (Not-a-Number)
 7. Operator Precedence
 8. Assignment Operators
 9. Unary Operators
 10. Key Takeaways
-

Introduction to Web Development

Web development is the process of creating websites and web applications that run in browsers. At its core, it involves writing code that browsers can understand and execute.

The Three Core Languages

Every website you visit is built using three fundamental technologies:

1. HTML (HyperText Markup Language)

- Provides the **structure** of web pages
- Defines content elements like headings, paragraphs, images, and links
- Think of it as the skeleton of a website

2. CSS (Cascading Style Sheets)

- Handles the **styling** and visual presentation
- Controls colors, fonts, layouts, and animations
- Think of it as the clothing and appearance

3. JavaScript (JS)

- Adds **behavior** and interactivity
- Enables dynamic content, user interactions, and real-time updates
- Think of it as the brain that makes things happen

Important Note

A website is essentially a collection of HTML, CSS, and JavaScript files, along with media assets like images, videos, and fonts. The browser downloads and interprets these files to display the web page.

Browser Console & REPL

The browser console is a built-in developer tool that allows you to write and test JavaScript code directly in your browser.

How to Open the Console

- **Windows/Linux:** Press `F12` or `Ctrl + Shift + J`
- **Mac:** Press `Cmd + Option + J`
- Alternatively: Right-click on any webpage → Select "Inspect" → Go to "Console" tab

REPL Pattern

The console follows a REPL (Read-Evaluate-Print-Loop) pattern:

1. **Read:** The console takes your input (JavaScript code)
2. **Evaluate:** It processes and executes the code
3. **Print:** It displays the result
4. **Loop:** It waits for your next input

Example

```
javascript
```

// Try this in the console:

1 + **2**

// Output: 3

5 * **10**

// Output: 50

"Hello" + **" World"**

// Output: "Hello World"

Why Use the Console?

- Quick testing of JavaScript code
 - Debugging errors in your code
 - Experimenting with new concepts
 - Inspecting variables and objects
-

Variables in JavaScript

A **variable** is a named container that stores data in your computer's memory. You can think of it as a labeled box where you keep information that you might need later.

The Box Analogy

Imagine variables as labeled boxes in a warehouse:

- The **box label** is the variable name (e.g., `age`, `name`)
- The **contents** of the box is the value (e.g., `23`, `"Tony Stark"`)
- You can put things in the box, take them out, or replace them

Declaring and Assigning Variables

javascript

```
// Creating a variable and assigning a value
```

```
let age = 23;  
let name = "Tony Stark";  
let isStudent = true;
```

```
// You can also declare without assigning (value will be undefined)
```

```
let score;  
console.log(score); // Output: undefined
```

```
// Later assign a value
```

```
score = 95;  
console.log(score); // Output: 95
```

Assignment Operator (=)

The `(=)` symbol is used to **assign** a value to a variable. It's NOT the same as mathematical equality.

```
javascript
```

```
let x = 10; // Assigns the value 10 to variable x
```

Variable Names - Best Practices

- Use meaningful names: `studentAge` instead of `sa`
- Start with a letter, underscore, or dollar sign
- Can contain letters, numbers, underscores, and dollar signs
- Cannot use reserved keywords (like `let`, `if`, `while`)
- Use camelCase for multi-word names: `firstName`, `totalScore`

Data Types

JavaScript has different types of data to represent different kinds of information. Let's explore the **primitive data types** - the basic building blocks.

1. Number

Represents numeric values, both integers and decimals.

```
javascript
```

```
let age = 23;           // Integer
let price = 99.99;      // Decimal (floating-point)
let temperature = -15;  // Negative number
let distance = 1.5e6;    // Scientific notation (1,500,000)
```

Important: JavaScript doesn't distinguish between integers and floats - they're all just "numbers".

2. String

Represents textual data. Strings must be enclosed in quotes.

```
javascript

let name = "Tony Stark";    // Double quotes
let greeting = 'Hello World'; // Single quotes
let message = `Welcome, ${name}`; // Template literals (backticks)

// Template literals allow embedded expressions
let age = 23;
let info = `I am ${age} years old`; // Output: "I am 23 years old"
```

3. Boolean

Represents logical values - only two possibilities: `true` or `false`.

```
javascript

let isStudent = true;
let isLoggedIn = false;
let hasPermission = true;

// Often used in conditions
if (isStudent) {
  console.log("Student discount applied!");
}
```

Common Uses:

- Toggle states (on/off, active/inactive)
- Conditional logic (if-else statements)
- Flags indicating status

4. Undefined

A variable that has been declared but not assigned a value.

javascript

```
let score;  
console.log(score);    // Output: undefined  
console.log(typeof score); // Output: "undefined"  
  
// This is different from not declaring a variable at all  
console.log(randomVar); // Error: randomVar is not defined
```

5. Null

Represents the **intentional absence** of any value. It's an assignment value meaning "no value" or "empty".

javascript

```
let data = null; // Explicitly saying "this has no value right now"  
  
// Common use case  
let user = null; // No user logged in yet  
  
// Later, when user logs in  
user = { name: "Alice", id: 101 };
```

Undefined vs Null:

- `undefined`: Variable exists but hasn't been assigned
- `null`: Deliberately set to "nothing"

6. BigInt

Used for very large integers that exceed the safe integer limit.

javascript

```
let bigNumber = 9007199254740991n; // Note the 'n' at the end  
let huge = 12345678901234567890n;
```

7. Symbol

Creates unique identifiers. Advanced concept - we'll cover later.

javascript

```
let id1 = Symbol('id');
let id2 = Symbol('id');
console.log(id1 === id2); // false - each symbol is unique
```

Checking Data Types

Use the `typeof` operator:

```
javascript

console.log(typeof 25);      // "number"
console.log(typeof "hello"); // "string"
console.log(typeof true);    // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null);    // "object" (this is a JavaScript quirk!)
console.log(typeof 123n);    // "bigint"
```

Arithmetic Operations

JavaScript provides standard mathematical operators to perform calculations.

Basic Operators

1. Addition (+)

```
javascript

let sum = 5 + 3;      // 8
let result = 10 + 20 + 5; // 35
```

2. Subtraction (-)

```
javascript

let difference = 10 - 3; // 7
let result = 50 - 25;    // 25
```

3. Multiplication (*)

```
javascript

let product = 4 * 5;    // 20
let area = 10 * 20;     // 200
```

4. Division (/)

javascript

```
let quotient = 20 / 4; // 5
let half = 10 / 2; // 5
let decimal = 7 / 2; // 3.5
```

5. Modulo (%)

Returns the **remainder** of a division.

javascript

```
let remainder = 12 % 5; // 2 (12 ÷ 5 = 2 remainder 2)
let check = 10 % 3; // 1 (10 ÷ 3 = 3 remainder 1)
let even = 8 % 2; // 0 (no remainder - 8 is even)
```

// Practical use: Check if a number is even or odd

```
let number = 7;
if (number % 2 === 0) {
  console.log("Even");
} else {
  console.log("Odd"); // This will execute
}
```

6. Exponentiation (**)

Raises a number to a power.

javascript

```
let squared = 5 ** 2; // 25 (5 squared)
let cubed = 2 ** 3; // 8 (2 cubed)
let power = 10 ** 3; // 1000 (10 to the power of 3)
```

Combining Operations

javascript

```
let result = 5 + 3 * 2; // 11 (not 16!)
// Multiplication happens first (operator precedence)
```

```
let complex = (5 + 3) * 2; // 16
// Parentheses force addition first
```


String Concatenation with +

When used with strings, `+` joins them together:

```
javascript

let firstName = "Tony";
let lastName = "Stark";
let fullName = firstName + " " + lastName; // "Tony Stark"

let age = 23;
let message = "I am " + age + " years old"; // "I am 23 years old"
```

NaN (Not-a-Number)

`NaN` is a special value in JavaScript that represents an **invalid or undefined numerical result**.

What Produces NaN?

```
javascript

// Dividing zero by zero
let result1 = 0 / 0;    // NaN

// Invalid mathematical operations
let result2 = Math.sqrt(-1); // NaN (square root of negative)

// Converting non-numeric strings to numbers
let result3 = Number("abc"); // NaN
let result4 = parseInt("hello"); // NaN

// Operations with undefined
let x;
let result5 = x * 5;    // NaN
```

Important Characteristics

1. NaN Propagates

Any operation involving `NaN` results in `NaN`:

```
javascript
```

```
let x = NaN;
console.log(x + 1); // NaN
console.log(x * 5); // NaN
console.log(x / 2); // NaN
console.log(x - 10); // NaN
```

2. Type of NaN is "number"

```
javascript

console.log(typeof NaN); // "number"
// This is a quirk of JavaScript!
```

3. NaN is Not Equal to Itself

```
javascript

console.log(NaN === NaN); // false
console.log(NaN == NaN); // false

// To check if a value is NaN, use:
console.log(isNaN(NaN)); // true
console.log(Number.isNaN(NaN)); // true (more reliable)
```

Checking for NaN

```
javascript

let value = 0 / 0;

// Method 1: isNaN() - converts value to number first
console.log(isNaN(value)); // true

// Method 2: Number.isNaN() - doesn't convert (recommended)
console.log(Number.isNaN(value)); // true

// Why Number.isNaN() is better:
console.log(isNaN("hello")); // true (converts "hello" to number first)
console.log(Number.isNaN("hello")); // false (doesn't convert)
```

Practical Tip

Always validate user input and handle potential NaN scenarios in your code to avoid unexpected bugs!

Operator Precedence

When an expression has multiple operators, JavaScript follows a specific order of evaluation, similar to **BODMAS/PEMDAS** in mathematics.

Order of Precedence (Highest to Lowest)

1. **Parentheses** `()`
2. **Exponentiation** `**`
3. **Multiplication** `*`, **Division** `/`, **Modulo** `%`
4. **Addition** `+`, **Subtraction** `-`

Examples

```
javascript
```

// Example 1: Multiplication before addition

```
let result1 = 5 + 3 * 2;
```

*// Step 1: $3 * 2 = 6$*

// Step 2: $5 + 6 = 11$

```
console.log(result1); // 11
```

// Example 2: Using parentheses to change order

```
let result2 = (5 + 3) * 2;
```

// Step 1: $(5 + 3) = 8$

*// Step 2: $8 * 2 = 16$*

```
console.log(result2); // 16
```

// Example 3: Complex expression

```
let result3 = (5 + 2) / 7 + 1 * 2;
```

// Step 1: $(5 + 2) = 7$

// Step 2: $7 / 7 = 1$

*// Step 3: $1 * 2 = 2$*

// Step 4: $1 + 2 = 3$

```
console.log(result3); // 3
```

// Example 4: Exponentiation has high precedence

```
let result4 = 2 + 3 ** 2;
```

*// Step 1: $3 ** 2 = 9$*

// Step 2: $2 + 9 = 11$

```
console.log(result4); // 11
```

// Example 5: Same precedence, left to right

```
let result5 = 10 / 2 * 3;
```

// Step 1: $10 / 2 = 5$ (left to right)

*// Step 2: $5 * 3 = 15$*

```
console.log(result5); // 15
```

Key Rules

1. Use **parentheses** when in doubt - they make code clearer
2. **Same precedence** operators evaluate **left to right**
3. **Exponentiation** is **right-to-left**: $2 ** 3 ** 2 = (2 ** (3 ** 2)) = (2 ** 9) = 512$

Pro Tip

When writing complex expressions, use parentheses liberally to make your intentions clear, even if they're not technically required:

```
// Less clear
```

```
let total = price + price * taxRate;
```

```
// More clear
```

```
let total = price + (price * taxRate);
```

Assignment Operators

Assignment operators provide shorthand ways to update variable values.

Basic Assignment (=)

```
javascript
```

```
let x = 10; // Assigns 10 to x
```

Compound Assignment Operators

Addition Assignment (+=)

```
javascript
```

```
let age = 20;
```

```
age += 5; // Same as: age = age + 5
```

```
console.log(age); // 25
```

```
// Equivalent to:
```

```
let score = 100;
```

```
score = score + 50; // 150
```

```
// vs,
```

```
score += 50; // 150 (shorter!)
```

Subtraction Assignment (-=)

```
javascript
```

```
let balance = 1000;
```

```
balance -= 200; // Same as: balance = balance - 200
```

```
console.log(balance); // 800
```

Multiplication Assignment (*=)

```
javascript
```

```
let price = 50;
price *= 2; // Same as: price = price * 2
console.log(price); // 100
```

Division Assignment (/=)

```
javascript

let total = 100;
total /= 4; // Same as: total = total / 4
console.log(total); // 25
```

Modulo Assignment (%=)

```
javascript

let number = 17;
number %= 5; // Same as: number = number % 5
console.log(number); // 2
```

Exponentiation Assignment (**=)

```
javascript

let base = 2;
base **= 3; // Same as: base = base ** 3
console.log(base); // 8
```

When to Use

These operators make code more concise and readable, especially in loops and calculations:

```
javascript
```

```
// Accumulating a total
let total = 0;
total += 100; // Add first item
total += 250; // Add second item
total += 75; // Add third item
console.log(total); // 425

// Doubling a value multiple times
let value = 5;
value *= 2; // 10
value *= 2; // 20
value *= 2; // 40
```

Unary Operators

Unary operators work with a **single operand** (unlike binary operators like $\oplus$ or $\ominus$ which need two operands).

Increment Operator (++)

Increases a value by 1.

Post-increment (variable++)

Uses the **current value first**, then increments.

```
javascript

let count = 5;
let result = count++;

console.log(result); // 5 (uses old value)
console.log(count); // 6 (now incremented)

// Step by step:
// 1. result = count (result gets 5)
// 2. count = count + 1 (count becomes 6)
```

Pre-increment (++variable)

Increments first, then uses the new value.

```
javascript
```

```
let count = 5;
let result = ++count;

console.log(result); // 6 (uses new value)
console.log(count); // 6 (incremented)

// Step by step:
// 1. count = count + 1 (count becomes 6)
// 2. result = count (result gets 6)
```

Comparison Example

```
javascript

let x = 10;
let a = x++; // Post-increment
console.log(a); // 10 (old value)
console.log(x); // 11 (incremented)

let y = 10;
let b = ++y; // Pre-increment
console.log(b); // 11 (new value)
console.log(y); // 11 (incremented)
```

Decrement Operator (--)

Decreases a value by 1. Works the same way as increment.

Post-decrement (variable--)

```
javascript

let score = 10;
let result = score--;

console.log(result); // 10 (old value)
console.log(score); // 9 (decremented)
```

Pre-decrement (--variable)

```
javascript
```



```
let score = 10;
let result = --score;

console.log(result); // 9 (new value)
console.log(score); // 9 (decremented)
```

Practical Example: The Tricky Case

```
javascript

let num = 5;
let newNum = num++; // newNum = 5, num = 6
newNum = ++num;    // num = 7, newNum = 7

console.log(num); // 7
console.log(newNum); // 7

// Step by step:
// Line 2: newNum gets 5 (post-increment), then num becomes 6
// Line 3: num becomes 7 (pre-increment), then newNum gets 7
```

Common Use Cases

1. Loop Counters

```
javascript

let i = 0;
while (i < 5) {
  console.log(i);
  i++; // Increment after each iteration
}
```

2. Counting Down

```
javascript

let countdown = 10;
while (countdown > 0) {
  console.log(countdown);
  countdown--; // Decrement
}
console.log("Blast off!");
```

3. Simple Incrementing

```
javascript  
  
let pageViews = 100;  
pageViews++; // Now 101
```

Pro Tips

1. **Avoid confusion:** Don't use increment/decrement in complex expressions

```
javascript  
  
// Bad (confusing)  
let x = 5;  
let y = x++ + ++x; // Hard to read!  
  
// Good (clear)  
let x = 5;  
x++;  
x++;  
let y = x + x;
```

2. **Use standalone:** Best used on their own line for clarity

```
javascript  
  
// Preferred  
let count = 10;  
count++;  
console.log(count);  
  
// Avoid (unless you have a good reason)  
let result = count++ + ++count;
```

Key Takeaways

Essential Concepts to Remember

1. **Variables are containers** for storing data values
2. **JavaScript has 7 primitive data types:** Number, String, Boolean, Undefined, Null, BigInt, Symbol
3. **typeof operator** tells you the data type of a value

4. **Arithmetic operators** perform mathematical calculations
5. **NaN** represents invalid numerical operations and propagates through calculations
6. **Operator precedence** determines the order of evaluation (remember BODMAS)
7. **Assignment operators** (`+=`, `-=`, etc.) provide shortcuts for updating variables
8. **Increment/Decrement** operators have pre and post variations that behave differently

Practice Makes Perfect

The concepts covered today form the **foundation** of JavaScript programming. Make sure to:

- Practice writing code in the browser console
- Experiment with different operators
- Try solving the assignment problems
- Ask questions when something is unclear

Common Beginner Mistakes to Avoid

1. **Confusing `=` with `==` or `===`**
 - `=` assigns a value
 - `==` and `===` compare values (we'll cover this later)
 2. **Forgetting operator precedence**
 - Use parentheses when in doubt!
 3. **Not understanding undefined vs null**
 - `undefined` = not assigned yet
 - `null` = intentionally empty
 4. **Mixing up pre and post increment**
 - `++x` increments first, then uses value
 - `x++` uses value first, then increments
-

Next Steps

In the upcoming sessions, we'll cover:

- **Conditional statements** (if-else)

- **Comparison operators** (==, ===, !=, <, >)
- **Logical operators** (&&, ||, !)
- **Loops** (for, while)
- **Functions**
- **Arrays and Objects**

Keep practicing and happy coding! 🚀

Quick Reference Card

javascript

// Variables

let age = 23;

const PI = 3.14;

// Data Types

typeof 42 // "number"

typeof "hello" // "string"

typeof true // "boolean"

typeof undefined // "undefined"

typeof null // "object" (quirk)

// Arithmetic

10 + 5 // 15

10 - 5 // 5

10 * 5 // 50

10 / 5 // 2

10 % 3 // 1

2 ** 3 // 8

// Assignment

x += 5 // x = x + 5

x -= 5 // x = x - 5

x *= 5 // x = x * 5

x /= 5 // x = x / 5

// Increment/Decrement

x++ // x = x + 1 (post)

++x // x = x + 1 (pre)

x-- // x = x - 1 (post)

--x // x = x - 1 (pre)

// Checking for NaN

isNaN(value) // true if NaN

Number.isNaN(value) // better method

Remember: The best way to learn programming is by doing. Open your browser console and start experimenting with the concepts you learned today!